



What is Name Mangling and How Does it Work?

In a previous lesson, you learned about prefixing attributes with a single underscore and a double underscore.

To remind you of the difference between them, a single underscore is a convention that means the attribute is meant for internal use in the class and should not be directly accessed from outside the class. Double underscore, on the other hand, prevents that attribute from being accessed directly from outside the class.

Here's an example that demonstrates how the two work:

```
class Example:
    def __init__(self):
        self._internal = 'I can be accessed from outside the class, but should not'
        self.__private = 'You cannot access me directly from outside the class'

obj = Example()

print(obj._internal) # I can be accessed from outside the class, but should not
print(obj.__private) # AttributeError: 'Example' object has no attribute '__private'
```

Prefixing an attribute with a double underscore triggers Python's name mangling process, in which Python internally renames the attribute by adding an underscore and the class name as a prefix, turning `__attribute` into `_ClassName__attribute`.

To see this in action, you create an instance of the class and use the `__dict__` special attribute of that instance, which is a dictionary containing the object's attributes:

```
class Example:
    def __init__(self, internal, private):
        self._internal = internal
        self.__private = private

example1 = Example(
    'I can be accessed from outside the class, but should not',
    'I cannot be accessed directly from outside the class'
)

print(example1.__dict__)
```

The result would be:

```
{
  '_internal': 'I can be accessed from outside the class, but should not',
  '_Example__private': 'I cannot be accessed directly from outside the class'
}
```

As you can see, the `__private` attribute is stored as `_Example__private`. This means you can still access that attribute outside the class this way:



Python Certification

Understanding Inheritance and Polymorphism

```
self._internal = internal
self.__private = private
```

```
example1 = Example(
    'I can be accessed from outside the class, but should not',
    'I cannot be accessed directly from outside the class'
)
example2 = Example(
    'I should not be accessed from outside the class',
    'But I can be accessed from outside the class with name mangling'
)

print(example1._Example__private) # I cannot be accessed directly from outside the class
print(example2._Example__private) # But I can be accessed from outside the class with name mangling
```

So, why does Python do name mangling?

The main purpose of name mangling is to prevent accidental attribute and method overriding when you use inheritance. Here's an example that makes that clear:

```
class Parent:
    def __init__(self):
        self.__data = 'Parent data'

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__data = 'Child data'

c = Child()
print(c.__dict__) # {'_Parent__data': 'Parent data', '_Child__data': 'Child data'}
```

You can see that both the `Parent` class and the `Child` that inherits from it have their separate `__class__data` attributes. This is made possible with name mangling. Otherwise, the `Child` would have overwritten the Parent data by accident.

Here's what would have happened without allowing Python to do the name mangling, that is if you don't prefix the attributes in both classes with double underscore:

```
class Parent:
    def __init__(self):
        self.data = 'Parent data'

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.data = 'Child data'

c = Child()
print(c.__dict__) # {'data': 'Child data'}
```



Python Certification

Understanding Inheritance and Polymorphism

But if you're working with a class that will be inherited, you should use a double underscore so the attribute from the parent doesn't get overridden.

Questions

What is the difference between a single underscore and a double underscore?

A single underscore and a double underscore are treated the same way by Python.

A single underscore makes attributes completely private, while a double underscore makes them protected.

A single underscore prevents direct access, while a double underscore allows direct access.

A single underscore is just a convention, while a double underscore triggers name mangling.

What is name mangling?

A process in which Python converts all attributes into methods for easier access.

A process in which Python deletes attributes with a single underscore to keep them hidden.

A process in which Python changes `__attribute` into `_ClassName__attribute` to avoid accidental overriding in subclasses.

A process in which Python encrypts private data to make it inaccessible from outside the class.

What happens when you don't prefix attributes in a parent and child classes with a double underscore?

Both classes keep their own separate copies of the attribute without interfering with each other.

The child class completely overrides the parent class attribute, and the parent's data is lost.

The parent class attributes become read-only and cannot be changed by the child class.

Python raises an error because attributes must always be prefixed with a double underscore.

Check your answer

Ask for Help



Python Certification



Understanding Inheritance and Polymorphism